

Vorlesung

Theoretische Informatik

(07 Kontextfreie Sprachen)

Alois Heinz
Hochschule Heilbronn,
Max-Planck-Str. 39, 74081 Heilbronn
heinz@hs-heilbronn.de

21. April/22. Mai 2026

Nachteile regulärer (Typ-3) Grammatiken

- Bereits manche einfachen Sprachen sind nicht regulär erzeugbar (etwa $L = \{a^n b^n \mid n \in \mathbb{N}\}$),
- Einfache Rechenausdrücke oder Programmiersprachen sind nicht erzeugbar.

Wie kann das geändert werden?

Abhilfe: Erweitere die Mächtigkeit der zugelassenen Regeln.

Gewinn: Komplexere Sprachen können beschrieben werden.

Kosten: Entscheidbarkeitsprobleme (Wortproblem etc.) sind schwieriger zu lösen.

Bsp: $G = (\{a, b\}, \{S\}, \{S \rightarrow aSb \mid \epsilon\}, S)$ ist eine erweiterte Grammatik, welche L erzeugen kann:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow^* a^n S b^n \Rightarrow a^n b^n$$

Kontextfreie Grammatiken

Eine Grammatik

$$G = (\Sigma, N, P, S)$$

mit Terminalalphabet Σ , dem davon disjunkten Nonterminalalphabet N , der endlichen Produktionsmenge P ist kontextfrei über Σ , wenn gilt:

$$P \subseteq N \times (\Sigma \cup N)^*$$

Eine Sprache L heißt kontextfrei über Σ , falls es eine kontextfreie Grammatik G über Σ gibt mit $L = L(G)$. Die Klasse aller kontextfreien Sprachen über Σ ist kfS_Σ .

Man spricht auch von Typ-2-Grammatiken und Typ-2-Sprachen: $TYP2_\Sigma = kfS_\Sigma$

Beispiel: Arithmetische Ausdrücke

Regeln für arithmetische Ausdrücke E

1. Variablen und Konstanten werden durch Bezeichner a dargestellt: $E \rightarrow a$
2. Ein Ausdruck kann geklammert werden: $E \rightarrow (E)$
3. Zwei Ausdrücke können durch Operatoren verknüpft werden: $E \rightarrow EOE$
4. Operatorsymbole können erzeugt werden: $O \rightarrow + \mid - \mid * \mid /$

Daraus ergibt sich die Grammatik:

$G_{Arith} = (\{a, (,), +, -, *, /\}, \{E, O\}, P, E)$ mit den Regeln

$P = \{E \rightarrow a \mid EOE \mid (E), O \rightarrow + \mid - \mid * \mid /\}$

Eine Ableitung mit G_{Arith} ist:

$$\begin{aligned} E &\Rightarrow EOE \Rightarrow E * E \Rightarrow E * (E) \Rightarrow E * (EOE) \Rightarrow E * (E - E) \Rightarrow E * (E - (E)) \Rightarrow \\ &E * (E - (EOE)) \Rightarrow E * (E - (E * E)) \Rightarrow E * (E - ((E) * E)) \Rightarrow E * (E - ((EOE) * E)) \Rightarrow \\ &E * (E - ((E + E) * E)) \Rightarrow E * ((E) - ((E + E) * E)) \Rightarrow E * ((EOE) - ((E + E) * E)) \Rightarrow \\ &E * (((E)/E) - ((E + E) * E)) \Rightarrow E * (((EOE)/E) - ((E + E) * E)) \Rightarrow \\ &E * (((E - E)/E) - ((E + E) * E)) \Rightarrow^* a * (((a - a)/a) - ((a + a) * a)) \end{aligned}$$

Reguläre und Kontextfreie Sprachen

Satz. Sei Σ ein Alphabet. Dann gilt:

1. $|\Sigma| \leq 1 \implies REG_{\Sigma} = kfS_{\Sigma}$
2. $|\Sigma| \geq 2 \implies REG_{\Sigma} \subset kfS_{\Sigma}$

Beweis. (teilweise)

1. Falls $|\Sigma| = 0$ (also $\Sigma = \emptyset$) dann gibt es 2 Sprachen: $\emptyset$ und $\{\epsilon\}$, beide regulär,
 $REG_{\emptyset} = \{\emptyset, \{\epsilon\}\} = kfS_{\emptyset}$
Falls $|\Sigma| = 1$: (siehe Literatur)
2. Falls $|\Sigma| \geq 2$: Jede reguläre Grammatik ist kontextfrei, daher gilt: $REG_{\Sigma} \subseteq kfS_{\Sigma}$. Seien $a, b \in \Sigma$ mit $a \neq b$, dann ist $\{a^n b^n \mid n \in \mathbb{N}\} \in kfS_{\Sigma}$, aber $\{a^n b^n \mid n \in \mathbb{N}\} \notin REG_{\Sigma}$, somit $REG_{\Sigma} \subsetneq kfS_{\Sigma}$

□

Normalformen für Kontextfreie Grammatiken

- **Normalformen** schränken Formen der zugelassenen Regeln weiter ein, ohne die Sprache wesentlich zu beschränken.
- **Ziel:** Vereinfachung der Verfahren zum Umgang mit den Grammatiken, etwa der Algorithmen zur Lösung des Wortproblems.
- **Simple Vereinfachungen** der Regeln:
 1. Entfernung von **ϵ -Regeln**:
 $G = (\Sigma, N, P, S)$, kontextfrei, heißt ϵ -frei, falls P keine Regeln der Art $A \rightarrow \epsilon$, $A \in N$, enthält
 2. Entfernung **nutzloser Symbole**:
 $A \in N$ ist nützlich, wenn
 - (1) aus A ein Terminalwort abgeleitet werden kann und
 - (2) A in einem aus S ableitbaren Wort vorkommt
 3. Entfernung von **Kettenregeln**: Regeln der Art $A \rightarrow B$ mit $A, B \in N$

Entfernung von ϵ -Regeln (1)

Satz. Zu jeder kontextfreien Grammatik $G = (\Sigma, N, P, S)$ kann eine ϵ -freie kontextfreie Grammatik $G_{\epsilon\text{-frei}}$ konstruiert werden mit

$$L(G_{\epsilon\text{-frei}}) = L(G) \setminus \{\epsilon\}$$

Beweis. In 5 Schritten:

1. $N_1 := \{A \in N \mid A \rightarrow \epsilon\}$, linke Seiten von ϵ -Regeln
2. Bestimme alle Nichtterminale, aus denen ϵ ableitbar ist:

do { $N_2 := N_1$;
 $N_1 := N_1 \cup \{A \in N \mid A \rightarrow w, w \in N_1^*\}$;
} **while** ($N_1 \neq N_2$)

Jetzt gilt: $A \in N_2 \Leftrightarrow A \Rightarrow^* \epsilon$

Entfernung von ϵ -Regeln (2)

3. Regeln $A \rightarrow w_1 B w_2$ mit $B \in N_2$ (also $B \Rightarrow^* \epsilon$) werden ergänzt durch $A \rightarrow w_1 w_2$:

$P_1 := P$;

do { $P_2 := P_1$;

for (**each** $A \rightarrow w_1 B w_2 \in P_2$)

if ($B \in N_2$) $P_1 := P_1 \cup \{A \rightarrow w_1 w_2\}$;

} **while** ($P_1 \neq P_2$)

4. ϵ -Regeln werden entfernt: $P_2 := P_1 \setminus \{r \in P_1 \mid r = A \rightarrow \epsilon\}$

5. $G_{\epsilon\text{-frei}} := (\Sigma, N, P_2, S)$

Für die so konstruierte Grammatik gilt: $L(G_{\epsilon\text{-frei}}) = L(G) \setminus \{\epsilon\}$ $\square$

Entfernung nutzloser Symbole

Satz. Zu jeder kontextfreien Grammatik $G = (\Sigma, N, P, S)$ mit $L(G) \neq \emptyset$ kann eine äquivalente kontextfreie Grammatik $G'' = (\Sigma'', N'', P'', S)$ ohne nutzlose Symbole konstruiert werden. (Eine solche Grammatik heißt *reduziert*.)

Beweis. In 2 Schritten:

1. $N' \subseteq N$ mit Symbolen, aus denen ein Terminalwort ableitbar ist, wird bestimmt:

Initialisierung: $N' := \{A \in N \mid A \rightarrow w \in P, w \in \Sigma^*\}$

Erweiterung: $N' := N' \cup \{A\}$, falls $A \rightarrow X_1 \dots X_k \in P$ mit $X_i \in \Sigma \cup N'$

Iteration: Die Erweiterung wird wiederholt, bis N' unverändert bleibt

Ergebnis: $G' = (\Sigma, N', P', S)$ mit $P' := \{A \rightarrow \alpha \in P \mid A \in N'\}$

2. Bestimme $N'' \cup \Sigma'' \subseteq N' \cup \Sigma$, die aus S ableitbar sind:

Initialisierung: $N'' := \{S\}$, $\Sigma'' := \emptyset$

Erweiterung: Für jedes $A \in N''$ und $A \rightarrow \omega \in P'$ wird N'' um die Nichtterminale und Σ'' um die Terminale aus ω erweitert

Iteration: Die Erweiterung wird wiederholt, bis N'' und Σ'' unverändert bleiben

Ergebnis: $G'' = (\Sigma'', N'', P'', S)$ mit $P'' := \{A \rightarrow \omega \in P' \mid \omega \in (N'' \cup \Sigma'')^*\}$

G'' ist äquivalent zu G und enthält keine nutzlosen Symbole. $\square$

Entfernung von Kettenregeln

Satz. Aus einer ϵ -freien reduzierten kontextfreien Grammatik $G = (\Sigma, N, P, S)$ können alle Kettenregeln entfernt werden.

Beweis. In 3 Schritten:

1. **Zykelentfernung:** Gibt es Regeln $B_1 \rightarrow B_2, \dots, B_{k-1} \rightarrow B_k$ und $B_k \rightarrow B_1$, dann können $B_1, \dots, B_k$ aus N entfernt und durch ein neues B ersetzt werden. Alle diese B_i werden in allen Regeln durch B äquivalenzerhaltend ersetzt.
2. **Neuordnung:** Die Elemente aus N werden mit $A_1, \dots, A_s$ benannt, so dass gilt: Aus $A_i \rightarrow A_j \in P$ folgt $i < j$. Dies ist möglich wegen der Zyklfreiheit
3. **Ersetzen** von Regeln $A_i \rightarrow A_j$:
 for (i **from** $s - 1$ **downto** 1)
 for (**all** $j > i$ **with** $A_i \rightarrow A_j \in P$) {
 $P := P \setminus \{A_i \rightarrow A_j\};$
 for (**all** $A_j \rightarrow \omega \in P$)
 $P := P \cup \{A_i \rightarrow \omega\}$
 }

Die veränderte Grammatik weist nun keine der Kettenregeln mehr auf. $\square$

Chomsky-Normalform (1)

Eine kontextfreie Grammatik $G = (\Sigma, N, P, S)$ ist in **Chomsky-Normalform** oder **CNF**, falls gilt:

$$P \subseteq N \times ((N \circ N) \cup \Sigma)$$

Kontextfreie Regeln in Chomsky-Normalform haben eine der Formen

$$\begin{array}{lcl} A & \rightarrow & BC \quad \text{oder} \\ A & \rightarrow & a \end{array}$$

mit $A, B, C \in N$ und $a \in \Sigma$

Satz. *Zu jeder kontextfreien Grammatik $G = (\Sigma, N, P, S)$ mit $\epsilon \notin L(G)$ existiert eine äquivalente Grammatik in G' Chomsky-Normalform.*

Beweis. G sei o.B.d.A. ϵ -frei, reduziert und ohne Kettenregeln. Dann stören nur noch Regeln der Form

$$A \rightarrow X_1 X_2 \dots X_m, m \geq 2$$

mit $X_i \in N \cup \Sigma$. Diese kann man in 2 Schritten nach CNF bringen:

Chomsky-Normalform (2)

$$A \rightarrow X_1 X_2 \dots X_m, m \geq 2, X_i \in N \cup \Sigma$$

1. Ist ein $X_i = a \in \Sigma$, wird es in jeder rechten Regelseite durch ein neues Symbol A ersetzt, und $N := N \cup \{A\}$; $P := P \cup \{A \rightarrow a\}$
2. Die Nicht-CNF Regeln haben jetzt die Form

$$A \rightarrow B_1 B_2 \dots B_m, m \geq 3, B_i \in N$$

Jede solche Regel wird ersetzt durch eine eigene Regelmenge:

$$\begin{aligned} A &\rightarrow B_1 D_1 \\ D_1 &\rightarrow B_2 D_2 \\ &\vdots \\ D_{m-3} &\rightarrow B_{m-2} D_{m-2} \\ D_{m-2} &\rightarrow B_{m-1} B_m \end{aligned}$$

mit neuen Nichtterminalen $D_i, i \in \{1, \dots, m-2\}$.

Die resultierende Grammatik ist in CNF und äquivalent zu G . $\square$

Greibach-Normalform

Eine kontextfreie Grammatik $G = (\Sigma, N, P, S)$ ist in Greibach-Normalform oder GNF, falls gilt:

$$P \subseteq N \times (\Sigma \circ N^*)$$

Kontextfreie Regeln in Greibach-Normalform haben die Form

$$A \rightarrow aB_1 \dots B_m, m \geq 0$$

mit $A, B_i \in N$ und $a \in \Sigma$. (Für $m \leq 1$ ist es eine Typ-3 Regel.)

Satz. *Zu jeder kontextfreien Grammatik $G = (\Sigma, N, P, S)$ mit $\epsilon \notin L(G)$ existiert eine äquivalente Grammatik in G' Greibach-Normalform.*

O.B.

Wortproblem für Kontextfreie Sprachen

Satz. Das Wortproblem für eine kontextfreie Sprache L kann in Zeit $O(n^3)$ entschieden werden, wenn n die Länge des Wortes ist.

Beweis. (Skizze)

Sei $L = L(G)$ und $G = (\Sigma, N, P, S)$ in CNF, sowie $w = x_1 \dots x_n \in \Sigma^*$.

Die Menge der Symbole aus N , aus denen sich das Teilwort $x_i \dots x_j$ (mit $1 \leq i \leq j \leq n$) ableiten lässt, sei $N_{i,j}$. $w \in L(G)$ falls $S \in N_{1,n}$.

$$N_{i,j} = \begin{cases} \{A \mid A \rightarrow x_i \in P\} & \text{falls } i = j \\ \{A \mid A \rightarrow BC \in P, B \in N_{i,k}, C \in N_{k+1,j}, i \leq k < j\} & \text{sonst} \end{cases}$$

Jedes $N_{i,j}$ kann durch einen Bit-Vektor fester Länge ($|N|$) dargestellt werden.

Es werden $n + (n - 1) + \dots + 1 = n(n + 1)/2$ viele $N_{i,j}$ berechnet.

Bei jeder Berechnung treten höchstens $n - 1$ viele Aufteilungen auf.

Bei Benutzung dynamischer Programmierung folgt dann die Behauptung. $\square$

Bem.: Das beschriebene Verfahren ist bekannt als **CYK-Algorithmus**, benannt nach Cocke, Younger und Kasami.